

Research on Performance, Monitoring & Analytics Tools

1. Redis Caching

Introduction

Redis (Remote Dictionary Server) is an open-source, in-memory data storage system mainly used for caching, session management, and real-time applications. Redis stores data in RAM instead of disk storage, making it extremely fast compared to traditional databases.

In modern backend systems, Redis is widely used to improve performance and reduce database load.

Why Redis is Important

When many users access the same API repeatedly, the backend continuously queries MongoDB, which increases response time and server load.

Redis solves this issue by temporarily storing frequently accessed data in memory so the backend can return responses much faster.

How Redis Works

Without Redis

User Request → Backend → MongoDB → Response

Every request hits the database directly.

With Redis

User Request → Backend → Redis Cache

If data exists in cache:

Redis returns data immediately.

If data does not exist:

Backend fetches from MongoDB and stores it in Redis for future requests.

Benefits of Redis

1. Faster API Responses

Redis significantly reduces response time because data is fetched directly from memory.

2. Reduced Database Load

Repeated queries are minimized, protecting MongoDB from overload.

3. Improved Scalability

Applications can handle more users simultaneously.

4. Session Management

Redis can securely store login sessions and authentication tokens.

5. Real-Time Performance

Ideal for real-time applications such as chat systems, live notifications, and AI processing.

Use Cases in Raritone

Product Data Caching

Store frequently viewed products in Redis.

User Session Storage

Maintain authentication sessions securely.

AI Try-On Result Caching

Store generated try-on outputs temporarily for faster retrieval.

Recommendation Systems

Cache personalized recommendations.

Redis Installation

```
npm install redis
```

Challenges of Redis

- Data stored in RAM may increase memory usage
- Cache invalidation must be handled properly
- Additional infrastructure management required

2. Sentry

Introduction

Sentry is a real-time error monitoring and performance tracking platform used by developers to detect, analyze, and fix backend or frontend issues.

It helps teams identify bugs immediately after they occur in production environments.

Why Sentry is Important

In large applications, manually checking server logs is difficult.

Sentry automatically captures:

- Errors
- Exceptions
- API failures
- Crashes
- Performance bottlenecks

This improves debugging speed and system reliability.

How Sentry Works

When an error occurs:

1. Sentry captures the exception
2. Generates detailed logs
3. Sends alerts to developers
4. Displays issue analytics dashboard

Features of Sentry

1. Real-Time Error Tracking

Instant detection of application failures.

2. Stack Trace Analysis

Shows exact file and line number causing errors.

3. Performance Monitoring

Tracks slow APIs and backend bottlenecks.

4. Crash Reporting

Captures unexpected server crashes automatically.

5. Alert Notifications

Developers receive alerts through:

- Email
- Slack
- Discord
- Microsoft Teams

Use Cases in Raritone

API Failure Monitoring

Detect broken API routes.

Image Upload Error Tracking

Monitor Cloudinary upload failures.

Authentication Error Detection

Track JWT authentication issues.

Database Error Monitoring

Capture MongoDB connection failures.

Benefits

- Faster debugging
- Reduced downtime
- Better backend stability
- Improved user experience

Installation

```
npm install @sentry/node
```

Challenges

- Requires proper configuration
- Large projects may generate many alerts
- Monitoring costs increase with scale

3. Grafana

Introduction

Grafana is an open-source monitoring and visualization platform used for displaying backend metrics and infrastructure analytics through dashboards.

It helps developers and DevOps teams monitor application health in real time.

Why Grafana is Important

Backend systems generate massive amounts of logs and metrics.

Grafana converts these metrics into easy-to-understand visual dashboards.

Features of Grafana

1. Real-Time Monitoring

Track live backend metrics.

2. Visualization Dashboards

Display:

- CPU usage
- Memory usage
- API requests
- Response times
- Error rates

3. Alert Management

Automatic alerts when server metrics exceed thresholds.

4. Multiple Data Source Support

Grafana integrates with:

- Prometheus
- MongoDB
- MySQL
- PostgreSQL
- Elasticsearch

How Grafana Works

1. Backend generates metrics
2. Metrics collected by monitoring tools
3. Grafana visualizes data in dashboards

Use Cases in Raritone

Monitor Backend APIs

Track API request volume and failures.

Monitor AI Processing

Visualize try-on processing performance.

Database Monitoring

Track MongoDB performance.

Server Health Tracking

Monitor CPU and memory utilization.

Benefits

- Improved infrastructure visibility
- Better system diagnostics
- Faster issue detection
- Real-time analytics

Common Integrations

Prometheus

Most popular monitoring integration for Grafana.

Node Exporter

Collects Linux server metrics.

MongoDB Exporter

Tracks MongoDB database metrics.

Challenges

- Requires monitoring infrastructure setup
- Dashboard configuration can be complex
- Initial setup may take time

4. Firebase Analytics

Introduction

Firebase Analytics is a Google analytics platform used for tracking user behavior, application performance, and engagement metrics.

It helps developers understand how users interact with applications.

Why Firebase Analytics is Important

Analytics helps companies make data-driven decisions.

Understanding user activity improves:

- User experience
- Feature optimization
- Business growth

Features

1. Event Tracking

Track custom user actions.

2. User Engagement Analytics

Monitor active users and session duration.

3. Performance Insights

Analyze application speed and responsiveness.

4. Crash Reporting Integration

Integrates with Firebase Crashlytics.

5. Real-Time User Data

Provides live analytics dashboards.

Use Cases in Raritone

Track User Login Activity

Analyze authentication trends.

Monitor Try-On Usage

Track AI try-on engagement.

Product Interaction Tracking

Analyze most viewed products.

User Retention Analysis

Measure returning users.

Benefits

- Better product decisions
- Improved user engagement
- Real-time analytics
- Scalable cloud integration

Installation

```
npm install firebase
```

Challenges

- Requires Firebase setup
- Analytics data interpretation needed
- Large datasets may require advanced analysis

Conclusion

Redis, Sentry, Grafana, and Firebase Analytics are essential tools for building scalable, secure, and production-ready backend systems.

These tools improve:

- Performance
- Monitoring
- Analytics
- Error tracking
- Scalability
- User experience

For the Raritone backend project, integrating these technologies will significantly enhance reliability, speed, and maintainability of the system.